Лабораторна робота № 7

Діагностика моделей: відхилення, дисперсія, розбіжність даних і пріоритизація помилок

Інтелектуальний аналіз та візуалізація даних · 4 години · 4 бали · лекції 9–10

Постановка

Ви — аналітик сервісу, який читає рукописні цифри з бланків. Перші моделі навчали на **сканах**: 20 000 чітких цифр. Тепер клієнти надсилають **фото з телефона**, і модель на них помиляється значно частіше. У кожного варіанта — свій «телефон»: свої вади зйомки, яких вам ніхто не опише.

Попередній аналітик залишив **виробничу модель** і пішов. Вам треба:

1. **поставити діагноз** виробничій моделі: відхилення, дисперсія чи розбіжність даних;
2. **знайти і впорядкувати за впливом** причини помилок на телефонних фото;
3. **скласти план дій** з обмеженим бюджетом і перевірити його числами.

> **Головне правило лабораторної.** Кожне твердження у звіті підкріплюється числом, яке ви отримали в цьому ноутбуці. «Модель перенавчилась» без двох похибок поруч — не діагноз.

Частина	Що робите	Бали
1	відхилення і дисперсія: чотири вибірки, людський рівень, криві навчання, дії	1,5
2	різні розподіли: розклад похибки, аналіз 100 помилок, ліки з бюджетом, план, аналіз за частинами, критика звіту колеги	2,0
3	звіт і декларація про використання ШІ	0,5

---

0. Ваш варіант

Впишіть прізвище та номер у списку групи. Файл даних підтягнеться сам.

In [1]:
```
STUDENT_NAME = 'Іванів Христина'
LIST_NUMBER  = 4
```

In [2]:
```
assert STUDENT_NAME and LIST_NUMBER > 0, 'впишіть прізвище та номер у списку
VARIANT = (LIST_NUMBER - 1) % 30 + 1
print('варіант', VARIANT)
```

```
варіант 4
```

> Поряд із ноутбуком має лежати тека `дані/` з файлами `скани.npz`, `lr7_варіанти_студенту.json` і **файлом вашого варіанта** `lr7_варіант_NN.npz` (кожен варіант — окремий файл близько 5 МБ; завантажте свій із ВНС).

In [3]:
```
import importlib.util, subprocess, sys
REQUIRED = ['numpy', 'pandas', 'scipy', 'scikit-learn', 'matplotlib']
MODNAME = {'scikit-learn': 'sklearn'}
missing = [p for p in REQUIRED if importlib.util.find_spec(MODNAME.get(p, p)
if missing:
    print('встановлюю:', ', '.join(missing), '...')
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', *mi
print('усі пакети на місці')
```

```
усі пакети на місці
```

In [4]:
```
import warnings; warnings.filterwarnings('ignore')
import os, json, time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import ndimage as ndi
import sklearn
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score

DATA_DIR = 'дані'
assert os.path.isdir(DATA_DIR), 'тека «дані» має лежати поруч із ноутбуком -
C1, C2, C3, C4, GREY = '#2a78d6', '#eb6834', '#1baf7a', '#c0392b', '#8a8a8a'
pd.set_option('display.width', 200); pd.set_option('display.max_colwidth', 9
print('scikit-learn', sklearn.__version__, '| numpy', np.__version__)
```

```
scikit-learn 1.9.1 | numpy 2.5.3
```

Допоміжні функції

Їх не треба змінювати — лише розуміти, що вони роблять. Опис кожної — у рядку документації.

In [5]:
```
import numpy as np
from scipy import ndimage as ndi


def flat(X):
```

```
    """Зображення uint8 (n, 28, 28) → ознаки у [0, 1], форма (n, 784)."""
    return X.reshape(len(X), -1).astype(np.float32) / 255.0


def split_train_dev(n, n_dev, seed=7):
    """Індекси навчальної частини і навчально-валідаційної вибірки (training
    perm = np.random.default_rng(seed).permutation(n)
    return perm[n_dev:], perm[:n_dev]


def _rotate(im, deg):
    t = np.deg2rad(deg); mat = np.array([[np.cos(t), -np.sin(t)], [np.sin(t)
    c = np.array(im.shape) / 2 - 0.5
    return ndi.affine_transform(im, mat, offset=c - mat @ c, order=1, mode='


def distort(X, seed, angle=None, blur=None, lowres=None, background=None, co
    """Синтез «телефонних» фото з чистих цифр. Кожен параметр — пара (від, д
    для кожного зображення береться випадкове значення; None — властивість в
    share — частка зображень, до яких застосовують задані властивості (решта
      angle      — кут повороту, градуси (знак обирається випадково)
      blur       — сигма гаусового розмиття, пікселі
      lowres     — до якої роздільності стиснути і розтягнути назад, пікселі
      background — яскравість фону, 0..1;  contrast — множник яскравості штр
      glare      — сила відблиску (пляма, у якій штрихи «вигорають» до кольо
      noise      — стандартне відхилення шуму сенсора
    Повертає uint8 (n, 28, 28)."""
    rng = np.random.default_rng(seed); out = np.empty(X.shape, np.float32)
    yy, xx = np.mgrid[0:28, 0:28]
    u = lambda r: rng.uniform(*r)
    for k, im in enumerate(X.astype(np.float32) / 255):
        if rng.random() < share:
            if angle: im = _rotate(im, rng.choice([-1, 1]) * u(angle))
            if blur: im = ndi.gaussian_filter(im, u(blur))
            if lowres:
                L = int(round(u(lowres))); im = ndi.zoom(ndi.zoom(im, L / 28
            bg = u(background) if background else 0.0
            im = bg + (u(contrast) if contrast else 1.0) * im
            if glare:
                cy, cx = rng.uniform(8, 20, 2); sg = rng.uniform(3.5, 6.5)
                w = u(glare) * np.exp(-((yy - cy) ** 2 + (xx - cx) ** 2) / (
            if noise: im = im + rng.normal(0, u(noise), im.shape)
        out[k] = np.clip(im, 0, 1)
    return (out * 255).round().astype(np.uint8)


def image_stats(X):
    """Прості вимірювання кожного фото за пікселями — ними можна відбирати ф
    rows = []
    yy, xx = np.mgrid[0:28, 0:28]
    for im in X.astype(np.float32) / 255:
        p50, p99 = np.percentile(im, [50, 99]); ink = np.clip(im - p50, 0, N
        cy, cx = (ink * yy).sum() / m, (ink * xx).sum() / m
        mu20, mu02, mu11 = (ink * (xx - cx) ** 2).sum() / m, (ink * (yy - cy
        rows.append({'контраст': p99 - p50,
                     'різкість': float(ndi.laplace(ndi.gaussian_filter(im, 0
```

```
                    'нахил_осі': float(np.degrees(0.5 * np.arctan2(2 * mu11
                    'шум': float((im - ndi.median_filter(im, 3)).std()),
                    'частка_чорнила': float((im > p50 + 0.5 * (p99 - p50)).
    return rows


def make_canvas(photos, seed, blobs=(1, 3), lines=0):
    """Полотна 56×56: телефонне фото цифри в довільному місці; плями-завади
    і (за потреби) lines горизонтальних ліній бланка. Повертає полотна (floa
    r = np.random.default_rng(seed); n = len(photos)
    X = np.clip(0.1 + r.normal(0, 0.04, (n, 56, 56)), 0, 1).astype(np.float3
    yy, xx = np.mgrid[0:56, 0:56]
    for i in range(n):
        rr, cc = r.integers(0, 29, 2); pos[i] = (rr, cc)
        X[i, rr:rr + 28, cc:cc + 28] = np.maximum(X[i, rr:rr + 28, cc:cc + 2
        for _ in range(r.integers(blobs[0], blobs[1] + 1)):
            while True:
                br, bc = r.integers(2, 54, 2)
                if not (rr - 3 <= br <= rr + 30 and cc - 3 <= bc <= cc + 30)
                    break
            s, a = r.uniform(1.2, 2.2), r.uniform(0.5, 0.9)
            X[i] = np.maximum(X[i], 0.1 + a * np.exp(-((yy - br) ** 2 + (xx
        for _ in range(lines):
            ly = r.integers(4, 52); X[i, ly:ly + 2, :] = np.maximum(X[i, ly:
    return np.clip(X, 0, 1), pos


def locate(X, thr=0.3):
    """Компонент А (без навчання): лівий верхній кут вирізу 28×28 навколо ци
    out = np.zeros((len(X), 2), int)
    for i, im in enumerate(X):
        ink = np.clip(ndi.gaussian_filter(im, 0.7) - thr, 0, None)
        s = ndi.uniform_filter(ink, size=28, mode='constant')[14:43, 14:43]
        r0, c0 = np.unravel_index(np.argmax(s), s.shape)
        w = ink[r0:r0 + 28, c0:c0 + 28].copy(); lab, n = ndi.label(w > 0)
        if n > 1:
            w[lab != 1 + np.argmax(ndi.sum(w, lab, range(1, n + 1)))] = 0
        m = w.sum() + 1e-9; cy = (w.sum(1) * np.arange(28)).sum() / m; cx =
        out[i] = (int(np.clip(round(r0 + cy - 14), 0, 28)), int(np.clip(rour
    return out


def crop(X, pos):
    """Вирізи 28×28 у форматі uint8, як фото."""
    return (np.stack([X[i, p[0]:p[0] + 28, p[1]:p[1] + 28] for i, p in enume


def shifted_copies(X, seed, k=2):
    """Копії зображень, зсунуті на випадкові ±k пікселів: класифікатор після
    r = np.random.default_rng(seed); out = np.empty_like(X)
    for i in range(len(X)):
        dy, dx = r.integers(-k, k + 1, 2); out[i] = np.roll(np.roll(X[i], dy
    return out
```

In [6]:

```
def err(model, X, y):
    """Похибка у відсотках (частка неправильних відповідей × 100)."""
```

```
    return round(100 * float((model.predict(X) != y).mean()), 2)


def show_grid(images, titles=None, ncol=10, size=1.05, title=None):
    """Сітка зображень; темне — чорнило."""
    n = len(images); nrow = (n + ncol - 1) // ncol
    fig, axs = plt.subplots(nrow, ncol, figsize=(size * ncol, size * nrow +
    for k, ax in enumerate(np.ravel(axs)):
        ax.axis('off')
        if k < n:
            ax.imshow(images[k], cmap='gray_r', vmin=0, vmax=255 if np.asarr
            if titles:
                ax.set_title(titles[k], fontsize=8)
    if title:
        fig.suptitle(title, fontsize=10)
    plt.tight_layout(); plt.show()


_TEST_CALLS = []


def evaluate_on_test(model, name):
    """Остаточна оцінка на тестовій вибірці. Викликайте ОДИН раз — для обран
    _TEST_CALLS.append(name)
    if len(_TEST_CALLS) > 1:
        print('УВАГА: тестову вибірку використано вже', len(_TEST_CALLS), 'р
    return err(model, flat(TEST_X), TEST_Y)
```

Дані

Масив	Що це	Скільки
`SCAN_X`, `SCAN_Y`	скани: чіткі цифри	20 000
`PHONE_X`, `PHONE_Y`	телефонні фото з мітками, для навчання	2000
`POOL_X`, `POOL_Y`	телефонні фото, які можна **докупити** (частина 2; бюджет — 1000 штук)	2000
`DEV_X`, `DEV_Y`	валідаційна вибірка: телефонні фото	1500
`TEST_X`, `TEST_Y`	тестова вибірка: телефонні фото — **лише для остаточної оцінки**	1500
`HUMAN_X`	50 фото для вимірювання людського рівня (мітки приховано)	50

Валідаційна і тестова вибірки — з того розподілу, на якому сервіс має працювати: телефонні фото (лекція 10, розділ 36).

In [7]:
```
S = np.load(os.path.join(DATA_DIR, 'скани.npz'))
V = np.load(os.path.join(DATA_DIR, 'lr7_варіант_%02d.npz' % VARIANT))
with open(os.path.join(DATA_DIR, 'lr7_варіанти_студенту.json'), encoding='ut
```

```
    INFO = json.load(f)[VARIANT - 1]
SCAN_X, SCAN_Y = S['scan_X'], S['scan_y']
PHONE_X, PHONE_Y = V['phone_train_X'], V['phone_train_y']
POOL_X, POOL_Y = V['pool_X'], V['pool_y']
DEV_X, DEV_Y = V['dev_X'], V['dev_y']
TEST_X, TEST_Y = V['test_X'], V['test_y']
HUMAN_X, HUMAN_CODE = V['human_X'], V['human_code']
for nm in ['SCAN_X', 'PHONE_X', 'POOL_X', 'DEV_X', 'TEST_X', 'HUMAN_X']:
    print('%-8s' % nm, globals()[nm].shape)
print()
print('Виробнича модель попереднього аналітика:')
print('  навчена на:', INFO['виробнича']['навчена_на'])
print('  модель:    ', INFO['виробнича']['модель'])
```

```
SCAN_X   (20000, 28, 28)
PHONE_X  (2000, 28, 28)
POOL_X   (2000, 28, 28)
DEV_X    (1500, 28, 28)
TEST_X   (1500, 28, 28)
HUMAN_X  (50, 28, 28)

Виробнича модель попереднього аналітика:
  навчена на: суміш
  модель:     make_pipeline(PCA(8, random_state=0), LogisticRegression(max_i
ter=300))
```

In [8]:
```
show_grid(SCAN_X[:10], [str(v) for v in SCAN_Y[:10]], title='скани')
show_grid(PHONE_X[:20], [str(v) for v in PHONE_Y[:20]], title='телефонні фот
```



Частина 1. Відхилення і дисперсія (1,5 бала)

---

1. Чотири вибірки

Щоб розрізнити дисперсію і розбіжність даних, потрібна **навчально-валідаційна вибірка** (training dev set): приклади з **того самого розподілу, на якому вчилася модель**, але не показані їй під час навчання (лекція 10, розділ 40).

Виробнича модель навчалась на джерелі, назва якого надрукована вище. Оберіть те саме джерело і відкладіть із нього навчально-валідаційну вибірку:

джерело	що це	скільки відкласти
`'скани'`	лише 20 000 сканів	2000
`'телефон'`	лише 2000 телефонних фото	400
`'суміш'`	20 000 сканів + 2000 фото	2000

Завдання 1.1

In [9]:
```
# ----- ВАШІ ВІДПОВІДІ -----
TRAIN_SOURCE = 'суміш'     # 'скани' / 'телефон' / 'суміш'
# чому навчально-валідаційну вибірку треба брати саме звідси, а не з телефон
WHY_TRAIN_DEV = 'Виробнича модель навчена на суміші (20 000 сканів + 2000 фо
# ----- КІНЕЦЬ -----
```

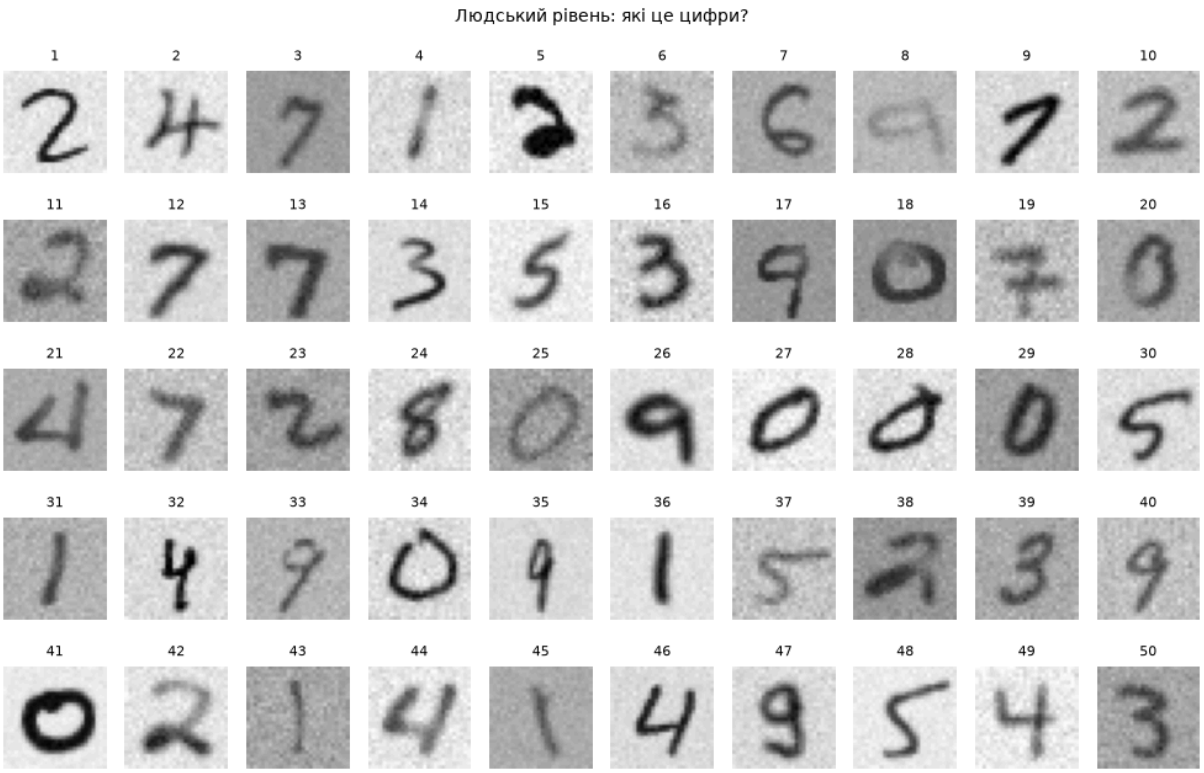
In [10]:
```
SOURCES = {'скани': (SCAN_X, SCAN_Y), 'телефон': (PHONE_X, PHONE_Y),
           'суміш': (np.concatenate([SCAN_X, PHONE_X]), np.concatenate([SCAN
N_TRAIN_DEV = {'скани': 2000, 'телефон': 400, 'суміш': 2000}
assert TRAIN_SOURCE in SOURCES, 'оберіть одне з трьох джерел'
assert WHY_TRAIN_DEV, 'поясніть вибір'
SRC_X, SRC_Y = SOURCES[TRAIN_SOURCE]
idx_tr, idx_td = split_train_dev(len(SRC_Y), N_TRAIN_DEV[TRAIN_SOURCE])
X_tr, y_tr = flat(SRC_X[idx_tr]), SRC_Y[idx_tr]
X_td, y_td = flat(SRC_X[idx_td]), SRC_Y[idx_td]
X_dev, y_dev = flat(DEV_X), DEV_Y
print(f'навчальна: {len(y_tr)} | навчально-валідаційна: {len(y_td)} | валіда
```

```
навчальна: 20000 | навчально-валідаційна: 2000 | валідаційна: 1500 | тестов
а: 1500 (не чіпаємо)
```

2. Людський рівень

Оптимальну похибку ми не знаємо, тож оцінюємо її **людським рівнем** — найкращим, якого досягає людина на тих самих фото (лекція 10, розділи 33–34). Подивіться на 50 фото свого варіанта і впишіть цифри підряд — 50 символів, пробіли не важливі. Не збільшуйте зображення більше, ніж його бачить модель, і не підглядайте в мітки.

In [11]:
```
show_grid(HUMAN_X, [str(k + 1) for k in range(50)], size=1.0, title='Людськи
```



Завдання 2.1

In [12]:
```
# ----- ВАШІ ВІДПОВІДІ -----
HUMAN_ANSWERS = '2471236972 2773539070 4728090005 1490915239 0214149543'
# ----- КІНЕЦЬ -----
```

In [13]:
```
ans = [int(c) for c in HUMAN_ANSWERS if c.isdigit()]
assert len(ans) == 50, f'потрібно 50 цифр, а введено {len(ans)}'
truth = (HUMAN_CODE - 7 * np.arange(50)) % 10
HUMAN_ERR = round(100 * float(np.mean(np.array(ans) != truth)), 1)
print('ваша похибка на 50 фото:', HUMAN_ERR, '%')
print('номери фото, де ви помилились:', [k + 1 for k in range(50) if ans[k]
```

```
ваша похибка на 50 фото: 6.0 %
номери фото, де ви помилились: [24, 30, 40]
```

> **Одна помилка на 50 фото — це 2 %, і довірчий інтервал такої оцінки широкий.** Як людський рівень беріть найкращий результат, якого досягла людина на цих фото: якщо одногрупник із тим самим варіантом має менше помилок — використайте його число і зазначте це у звіті.

3. Діагноз виробничої моделі

Відтворюємо виробничу модель і рахуємо похибки на трьох вибірках. Розклад (лекція 10, розділ 41):

$$\underbrace{E_{\text{навч}} - E_{\text{людина}}}_{\text{усувне відхилення}}, \qquad \underbrace{E_{\text{навч-вал}} - E_{\text{навч}}}_{\text{дисперсія}}, \qquad \underbrace{E_{\text{вал}} - E_{\text{навч-вал}}}_{\text{розбіжність даних}}$$

In [14]:
```
PRODUCTION = eval(INFO['виробнича']['модель'])        # модель саме в тому в
t0 = time.time()
prod = PRODUCTION.fit(X_tr, y_tr)
E_PROD = {'навч': err(prod, X_tr, y_tr), 'навч_вал': err(prod, X_td, y_td),
print(f'навчено за {time.time() - t0:.0f} с')
print('похибки виробничої моделі, %:', E_PROD)
DECOMP_PROD = {'усувне відхилення': round(E_PROD['навч'] - HUMAN_ERR, 2),
               'дисперсія': round(E_PROD['навч_вал'] - E_PROD['навч'], 2),
               'розбіжність даних': round(E_PROD['вал'] - E_PROD['навч_вал']
pd.Series(DECOMP_PROD, name='п. п.').to_frame()
```

```
навчено за 0 с
похибки виробничої моделі, %: {'навч': 25.4, 'навч_вал': 23.9, 'вал': 32.2}
```

Out[14]:

	п. п.
усувне відхилення	19.4
дисперсія	-1.5
розбіжність даних	8.3

Завдання 3.1

Поставте діагноз. Якщо дві складові близькі, назвіть головну і поясніть, чому друга — другорядна.

In [15]:
```
# ----- ВАШІ ВІДПОВІДІ -----
DIAGNOSIS = 'відхилення'        # 'відхилення' / 'дисперсія' / 'розбіжність'
# доказ числами (дві похибки поруч і різниця між ними):
DIAGNOSIS_WHY = 'Навчальна 25,4 %, навч–вал 23,9 %, валідаційна 32,2 %, люди
# ----- КІНЕЦЬ -----
```

In [16]:
```
assert DIAGNOSIS in ('відхилення', 'дисперсія', 'розбіжність') and DIAGNOSIS
print('діагноз:', DIAGNOSIS)
```

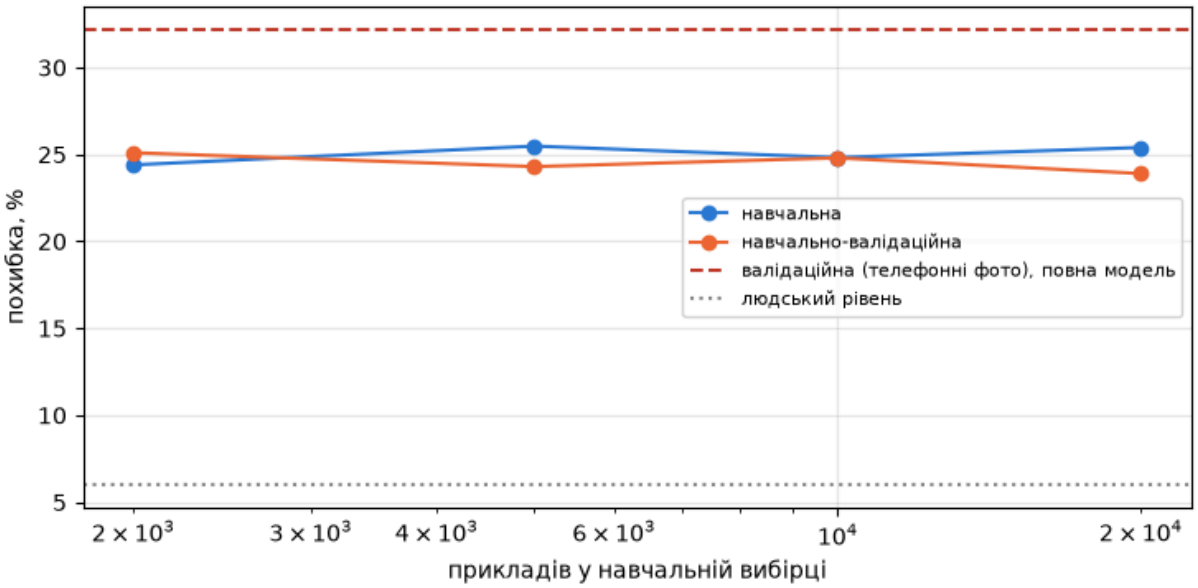
```
діагноз: відхилення
```

4. Крива навчання

Крива навчання показує, чи допоможуть **ще дані того самого розподілу** (лекція 9, розділи 28–30). Навчаємо виробничу модель на 10, 25, 50 і 100 % навчальної частини і дивимось на похибку на навчальній і навчально-валідаційній вибірках; пунктиром — валідаційна похибка повної моделі і ваш людський рівень. Це триває 1–3 хвилини.

In [17]:
```
curve = []
for frac in [0.1, 0.25, 0.5, 1.0]:
```

```
    n = max(50, int(len(y_tr) * frac))
    m = eval(INFO['виробнича']['модель']).fit(X_tr[:n], y_tr[:n])
    curve.append({'n': n, 'навч': err(m, X_tr[:n], y_tr[:n]), 'навч_вал': er
CURVE = pd.DataFrame(curve)
fig, ax = plt.subplots(figsize=(7.5, 3.8))
ax.plot(CURVE['n'], CURVE['навч'], 'o-', color=C1, label='навчальна')
ax.plot(CURVE['n'], CURVE['навч_вал'], 'o-', color=C2, label='навчально-валі
ax.axhline(E_PROD['вал'], ls='--', color=C4, label='валідаційна (телефонні ф
ax.axhline(HUMAN_ERR, ls=':', color=GREY, label='людський рівень')
ax.set_xscale('log'); ax.set_xlabel('прикладів у навчальній вибірці'); ax.se
ax.legend(fontsize=8); ax.grid(alpha=.3); plt.tight_layout(); plt.show()
CURVE
```



Out[17]:

	n	навч	навч_вал
0	2000	24.40	25.1
1	5000	25.48	24.3
2	10000	24.82	24.8
3	20000	25.40	23.9

Завдання 4.1

In [18]:
```
# ----- ВАШІ ВІДПОВІДІ -----
MORE_SAME_DATA_HELPS = False   # True / False: чи зменшать валідаційну похиб
# що на графіку це показує (форма двох кривих, розрив між ними, де валідаці
CURVE_WHY = 'Обидві криві плоскі й лежать разом високо: навчальна 24,4 → 25,
# ----- КІНЕЦЬ -----
```

In [19]:
```
assert MORE_SAME_DATA_HELPS in (True, False) and CURVE_WHY, 'заповніть висно
```

5. Дії: що допомагає, а що — ні

Таблиці лекції 9 (розділи 25 і 27) кажуть, які дії зменшують відхилення, а які — дисперсію; лекція 10 (розділи 37–43) — що робити з розбіжністю. Перевірте це на своїй моделі:

- **дві дії**, які, за вашим діагнозом, мають допомогти;
- **одну контрольну дію**, спрямовану на **іншу** проблему, — вона допомогти не повинна.

Кожна дія — це модель і навчальні дані. Похибки рахуються на тих самих навчально-валідаційній і валідаційній вибірках, що й у пункті 3. Готові «цеглинки»: `X_tr, y_tr` — навчальна частина вашого джерела; `flat(SCAN_X), SCAN_Y` — усі скани; `flat(PHONE_X), PHONE_Y` — усі телефонні фото (увага: частина з них — у навчально-валідаційній вибірці, якщо джерело `'телефон'` чи `'суміш'`; використовуйте `X_tr` і додавайте до нього лише те, чого в ній немає); `SRC_X[idx_tr]` — ті самі навчальні приклади як зображення, з яких `shifted_copies(SRC_X[idx_tr], seed, k)` робить копії, зсунуті на ±k пікселів (аугментація); `distort(...)` — синтез «телефонних» фото (опис — у розділі 8).

Приклад запису:

```
('більша модель: MLP(256)', 'відхилення',
 MLPClassifier(hidden_layer_sizes=(256,), max_iter=30,
random_state=0), X_tr, y_tr)
```

Одна дія навчається від кількох секунд до хвилини.

In [20]:
```
# ----- ВАШІ ВІДПОВІДІ -----
ACTIONS = [
    # (назва дії, на яку проблему спрямована, модель, навчальні X, навчальні
    ('більша модель: MLP(256)', 'відхилення',
     MLPClassifier(hidden_layer_sizes=(256,), max_iter=30, random_state=0),
    ('більше компонент: PCA(64)+LR', 'відхилення',
     make_pipeline(PCA(64, random_state=0), LogisticRegression(max_iter=300)
    ('контроль: сильніша регуляризація (C=0.01)', 'дисперсія',
     make_pipeline(PCA(8, random_state=0), LogisticRegression(C=0.01, max_it
]
# ----- КІНЕЦЬ -----
```

In [21]:
```
assert len(ACTIONS) >= 3, 'потрібно щонайменше три дії: дві за діагнозом і с
rows = [{'дія': 'виробнича модель (до змін)', 'на що спрямована': '—', **E_P
for name, target, model, Xa, ya in ACTIONS:
    t0 = time.time(); model.fit(Xa, ya)
    rows.append({'дія': name, 'на що спрямована': target, 'навч': err(model,
                 'навч_вал': err(model, X_td, y_td), 'вал': err(model, X_dev
    print(f'{name}: {time.time() - t0:.0f} с')
ACT_TABLE = pd.DataFrame(rows)
ACT_TABLE['Δ дисперсії, п. п.'] = (ACT_TABLE['навч_вал'] - ACT_TABLE['навч']
ACT_TABLE['Δ відхилення, п. п.'] = ACT_TABLE['навч'] - E_PROD['навч']
ACT_TABLE['Δ валідаційної, п. п.'] = ACT_TABLE['вал'] - E_PROD['вал']
ACT_TABLE.round(2)
```

```
більша модель: MLP(256): 4 с
```

```
більше компонент: PCA(64)+LR: 0 с
контроль: сильніша регуляризація (C=0.01): 0 с
```

Out[21]:

	дія	на що спрямована	навч	навч_вал	вал	Δ дисперсії, п. п.	Δ відхилення, п. п.	вал
0	виробнича модель (до змін)	—	25.40	23.9	32.20	0.00	0.00	
1	більша модель: MLP(256)	відхилення	0.77	4.4	14.13	5.13	-24.63	
2	більше компонент: PCA(64)+LR	відхилення	9.04	9.8	17.33	2.26	-16.36	
3	контроль: сильніша регуляризація (C=0.01)	дисперсія	26.02	25.5	32.67	0.98	0.62	

Завдання 5.1

In [22]:
```
# ----- ВАШІ ВІДПОВІДІ -----
# що допомогло, що ні і чому — числами з таблиці; чи справдився прогноз для
ACTIONS_CONCLUSION = 'Діагноз — відхилення, тож допомогти мали дві дії проти
# ----- КІНЕЦЬ -----
```

In [23]:
```
assert ACTIONS_CONCLUSION, 'сформулюйте висновок'
```

Частина 2. Різні розподіли і пріоритизація помилок (2,0 бала)

---

6. Модель B і розклад її похибки

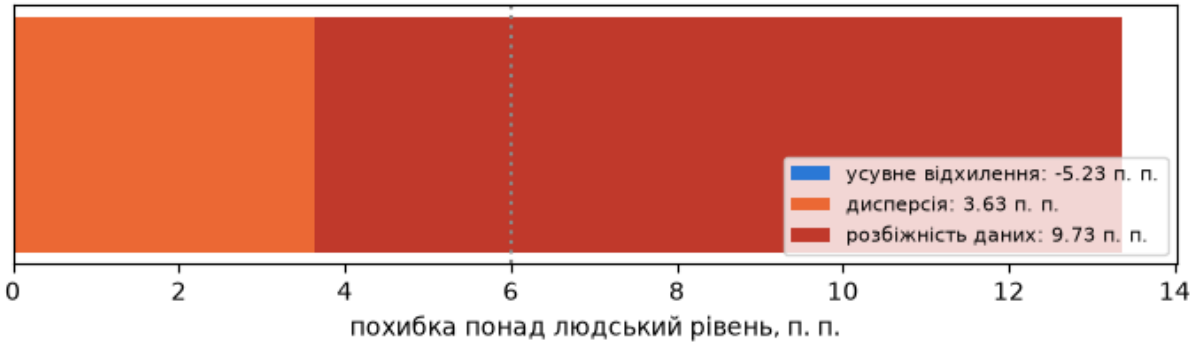
Далі працюємо з моделлю, яку радить лекція 10 (розділ 37): велика мережа на **суміші** сканів і телефонних фото. Навчально-валідаційна вибірка — 2000 прикладів суміші.

In [24]:
```
MIX_X, MIX_Y = SOURCES['суміш']
mix_tr, mix_td = split_train_dev(len(MIX_Y), 2000)
XB_tr, yB_tr = flat(MIX_X[mix_tr]), MIX_Y[mix_tr]
XB_td, yB_td = flat(MIX_X[mix_td]), MIX_Y[mix_td]
```

```
def model_b():
    return MLPClassifier(hidden_layer_sizes=(256,), max_iter=30, random_stat


t0 = time.time()
B = model_b().fit(XB_tr, yB_tr)
pred_B = B.predict(X_dev)
E_B = {'навч': err(B, XB_tr, yB_tr), 'навч_вал': err(B, XB_td, yB_td), 'вал'
print(f'навчено за {time.time() - t0:.0f} с; похибки моделі B, %:', E_B)
DECOMP_B = {'усувне відхилення': round(E_B['навч'] - HUMAN_ERR, 2), 'дисперс
            'розбіжність даних': round(E_B['вал'] - E_B['навч_вал'], 2)}
fig, ax = plt.subplots(figsize=(7, 2.2))
left = 0
for (k, v), c in zip(DECOMP_B.items(), [C1, C2, C4]):
    ax.barh(0, max(v, 0), left=left, color=c, label=f'{k}: {v} п. п.'); left
ax.axvline(HUMAN_ERR, color=GREY, ls=':'); ax.set_yticks([]); ax.set_xlabel(
ax.legend(fontsize=8, loc='lower right'); plt.tight_layout(); plt.show()
```

```
навчено за 4 с; похибки моделі B, %: {'навч': 0.77, 'навч_вал': 4.4, 'вал':
14.13}
```



Завдання 6.1

In [25]:
```
# ----- ВАШІ ВІДПОВІДІ -----
DIAGNOSIS_B = 'розбіжність'      # 'відхилення' / 'дисперсія' / 'розбіжність
# чому саме це і чому більше сканів тут не допоможе (або допоможе):
DIAGNOSIS_B_WHY = 'Модель B: навчальна 0,77 %, навч–вал 4,4 %, валідаційна 1
# ----- КІНЕЦЬ -----
```

In [26]:
```
assert DIAGNOSIS_B in ('відхилення', 'дисперсія', 'розбіжність') and DIAGNOS
```

7. Аналіз 100 помилок

Метод лекції 9 (розділи 14–19) і лекції 10 (розділ 42): переглянути помилки **очима**, позначити, що з кожним фото не так, і порахувати, скільки дасть усунення кожної причини.

Нижче — перші 100 помилок моделі B на валідаційній вибірці (у порядку номерів фото). Над кожним фото: порядковий номер помилки, **мітка → прогноз моделі**.

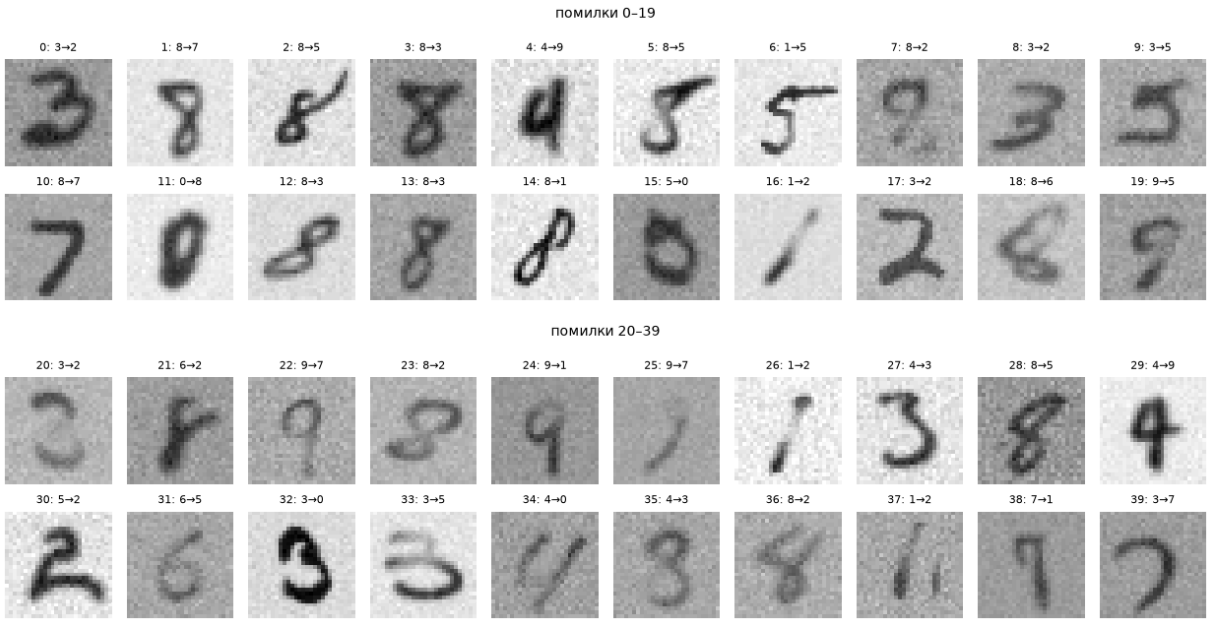
Словник категорій (можна кілька через `+`):

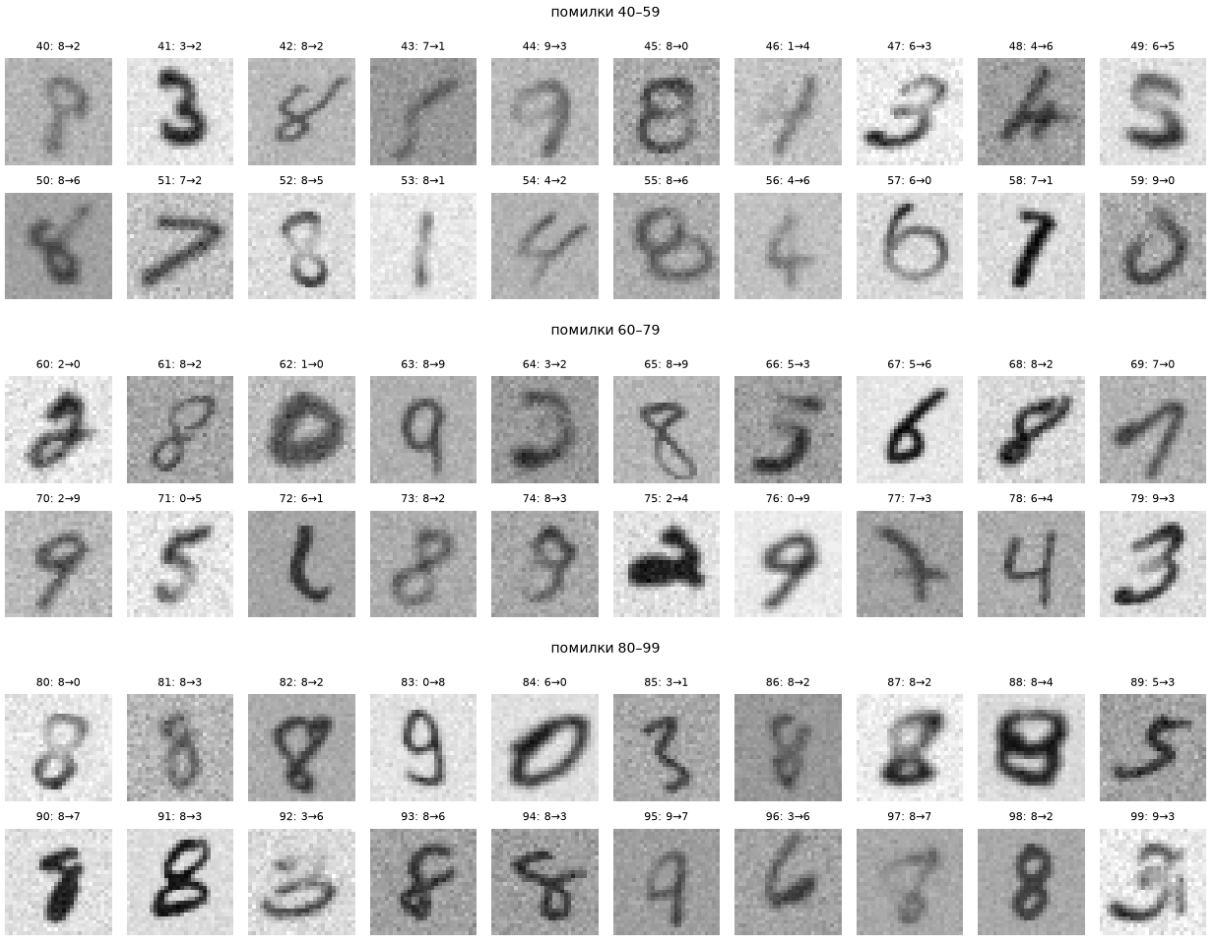
категорія	як виглядає
`нахил`	цифра помітно повернута
`розмиття`	штрихи розмиті, краї нечіткі
`низька роздільність`	«сходинки», цифра наче з кількох великих пікселів
`тьмяність`	сірий фон, блідий слабкий штрих
`відблиск`	частина штриха «вигоріла» до кольору фону
`шум`	зерно, крапки по всьому фото
`хибна мітка:N`	модель, схоже, права, а мітка хибна; N — правильна, на вашу думку, цифра
`нерозбірливо`	людина теж не впевнена
`інше`	нічого з переліченого

Приклад: `'нахил+шум'`, `'хибна мітка:4'`, `'інше'`.

In [27]:
```
ERR_IDX = np.where(pred_B != DEV_Y)[0][:100]
print('помилок моделі B на валідаційній вибірці:', int((pred_B != DEV_Y).sum
for page in range(0, len(ERR_IDX), 20):
    ids = ERR_IDX[page:page + 20]
    show_grid(DEV_X[ids], [f'{page + k}: {DEV_Y[i]}→{pred_B[i]}' for k, i in
              title=f'помилки {page}–{page + len(ids) - 1}')
```

```
помилок моделі B на валідаційній вибірці: 212 | переглядаємо: 100
```





Завдання 7.1

Заповніть по одному рядку на помилку, у тому самому порядку (якщо помилок менше 100 — лише стільки рядків, скільки помилок; зайві залиште порожніми). Це найдовша, але й найважливіша частина роботи: на ній стоїть увесь план.

In [28]:
```
# ----- ВАШІ ВІДПОВІДІ -----
TAGS = [
    'тьмяність', 'інше', 'нахил', 'тьмяність', 'інше', 'нахил', 'хибна мітка
    'хибна мітка:7', 'розмиття', 'нахил', 'тьмяність', 'нахил', 'тьмяність',
    'тьмяність', 'тьмяність', 'тьмяність', 'тьмяність+нахил', 'тьмяність', '
    'хибна мітка:2', 'тьмяність', 'інше', 'інше', 'тьмяність+нахил', 'хибна
    'тьмяність', 'інше', 'тьмяність+нахил', 'тьмяність', 'тьмяність', 'тьмян
    'тьмяність+розмиття', 'нахил', 'інше', 'хибна мітка:1', 'тьмяність+нахил
    'нахил', 'тьмяність', 'хибна мітка:0', 'хибна мітка:9', 'тьмяність', 'ть
    'хибна мітка:9', 'хибна мітка:5', 'тьмяність', 'тьмяність', 'тьмяність',
    'інше', 'тьмяність+шум', 'тьмяність', 'хибна мітка:9', 'хибна мітка:0',
    'інше', 'інше', 'інше', 'тьмяність', 'тьмяність', 'тьмяність', 'хибна мі
]
# ----- КІНЕЦЬ -----
```

In [29]:
```
CATS = ['нахил', 'розмиття', 'низька роздільність', 'тьмяність', 'відблиск',
assert len(TAGS) >= len(ERR_IDX), f'потрібно {len(ERR_IDX)} рядків, а у спис
TAGS = TAGS[:len(ERR_IDX)]
parsed, relabel = [], {}
```

```
for k, t in enumerate(TAGS):
    assert t.strip(), f'рядок {k} порожній'
    cs = []
    for part in t.split('+'):
        part = part.strip()
        if part.startswith('хибна мітка'):
            cs.append('хибна мітка')
            if ':' in part:
                relabel[int(ERR_IDX[k])] = int(part.split(':')[1])
        else:
            assert part in CATS, f'рядок {k}: невідома категорія «{part}»'
            cs.append(part)
    parsed.append(set(cs))
n = len(parsed)
ERR_TABLE = pd.DataFrame([{'категорія': c, 'помилок': sum(c in p for p in pa
ERR_TABLE['частка помилок, %'] = (100 * ERR_TABLE['помилок'] / n).round(1)
ERR_TABLE['стеля: на скільки п. п. впаде похибка'] = (E_B['вал'] * ERR_TABLE
ERR_TABLE = ERR_TABLE.sort_values('помилок', ascending=False).reset_index(dr
print('похибка моделі B на валідаційній:', E_B['вал'], '%; категорії можуть
ERR_TABLE
```

```
похибка моделі B на валідаційній: 14.13 %; категорії можуть перетинатись, то
ж сума часток > 100 %
```

Out[29]:

	категорія	помилок	частка помилок, %	стеля: на скільки п. п. впаде похибка
0	тьмяність	51	51.0	7.21
1	хибна мітка	22	22.0	3.11
2	нахил	15	15.0	2.12
3	інше	14	14.0	1.98
4	розмиття	5	5.0	0.71
5	шум	4	4.0	0.57
6	нерозбірливо	1	1.0	0.14
7	низька роздільність	0	0.0	0.00
8	відблиск	0	0.0	0.00

Хибні мітки. Якщо ви позначили фото як `хибна мітка:N`, нижче похибка моделі B за **виправленими** мітками валідаційної вибірки. Яка частина похибки — не провина моделі?

In [30]:
```
DEV_Y_FIXED = DEV_Y.copy()
for i, lab in relabel.items():
    DEV_Y_FIXED[i] = lab
E_B_FIXED = round(100 * float((pred_B != DEV_Y_FIXED).mean()), 2)
print(f'виправлено міток: {len(relabel)}; похибка B: {E_B["вал"]} % → {E_B_F
```

```
виправлено міток: 22; похибка B: 14.13 % → 12.73 % за виправленими мітками
```

Завдання 7.2

Упорядкуйте **до трьох** напрямів роботи за впливом — від найбільшої стелі до меншої. Для хибних міток рішення інше: їх не «лікують» моделлю, а перевіряють і виправляють розмітку (лекція 9, розділ 16).

In [31]:
```
# ----- ВАШІ ВІДПОВІДІ -----
PRIORITY = ['тьмяність', 'хибна мітка', 'нахил']
# чому саме такий порядок і що ви зробите з кожною категорією:
PRIORITY_WHY = '3 перших 100 помилок: тьмяність 51 (стеля 7,21 п. п. з 14,13
# ----- КІНЕЦЬ -----
```

In [32]:
```
assert 1 <= len(PRIORITY) <= 3 and all(p in CATS for p in PRIORITY) and PRIC
```

8. Ліки від розбіжності: бюджет — 1000 фото

Варіанти з лекції 10 (розділи 42–43):

1. **докупити** 1000 справжніх телефонних фото з пулу — випадкових;
2. **докупити** 1000 фото, **відібраних без міток**: за вимірюваннями пікселів (`image_stats`) або за невпевненістю моделі B (`B.predict_proba`);
3. **синтезувати** 20 000 «телефонних» фото зі сканів функцією `distort` — бюджету не потребує.

Купити можна лише **одну** тисячу. Кожен варіант навчається 30–60 с.

In [33]:
```
def train_with(extra_X, extra_y):
    """Модель B, навчена на суміші плюс додаткових даних; повертає модель і
    m = model_b().fit(np.concatenate([XB_tr, flat(extra_X)]), np.concatenate
    return m, err(m, X_dev, y_dev)


FIX = [{'ліки': 'база: модель B', 'вал': E_B['вал']}]
rand_idx = np.random.default_rng(11).permutation(len(POOL_Y))[:1000]
m_rand, e = train_with(POOL_X[rand_idx], POOL_Y[rand_idx])
FIX.append({'ліки': '+1000 випадкових фото з пулу', 'вал': e}); pd.DataFrame
```

Out[33]:

	ліки	вал
0	база: модель B	14.13
1	+1000 випадкових фото з пулу	11.07

Завдання 8.1. Цілеспрямована покупка

Відберіть 1000 фото з пулу, **не дивлячись на мітки** `POOL_Y` , — так, щоб серед них було якомога більше фото з категорії-пріоритету. Вимірювання кожного фото пулу вже пораховано:

In [34]:
```
POOL_STATS = pd.DataFrame(image_stats(POOL_X))
POOL_STATS['впевненість_B'] = B.predict_proba(flat(POOL_X)).max(1)
POOL_STATS.describe().round(3)
```

Out[34]:

	контраст	різкість	нахил_осі	шум	частка_чорнила	впевненість_B
count	2000.000	2000.000	2000.000	2000.000	2000.000	2000.000
mean	0.543	0.015	-7.472	0.041	0.128	0.849
std	0.176	0.008	27.829	0.009	0.039	0.201
min	0.145	0.003	-89.637	0.021	0.038	0.194
25%	0.392	0.009	-25.622	0.034	0.098	0.770
50%	0.508	0.013	-9.021	0.041	0.125	0.953
75%	0.704	0.018	8.775	0.047	0.152	0.995
max	0.895	0.064	88.665	0.080	0.290	1.000

Підказки: `контраст` — різниця між штрихом і фоном; `різкість` — дисперсія лапласіана (розмиті й низької роздільності фото мають малу); `шум` — залишок після медіанного фільтра; `частка_чорнила` — частка пікселів, яскравіших за середину між фоном і штрихом; `нахил_осі` — кут головної осі цифри (слабка ознака: цифри й самі бувають нахилені); `впевненість_B` — найбільша ймовірність, яку дає модель B (мала — модель сумнівається).

```
SELECT = POOL_STATS.sort_values('контраст').index[:1000].to_numpy()
# приклад: найменш контрастні
```

In [35]:
```
# ----- ВАШІ ВІДПОВІДІ -----
# відбір БЕЗ міток: тьмяні фото — це низький контраст, а також ті, у яких мо
# Береться 1000 фото з найменшою сумою рангів за «контраст» і «впевненість_B
_score = POOL_STATS['впевненість_B'].rank() + POOL_STATS['контраст'].rank()
SELECT = _score.sort_values().index[:1000].to_numpy()          # масив із 10
SELECT_WHY = 'Пріоритет — тьмяність (слабкий штрих на сірому тлі). Без міток
# ----- КІНЕЦЬ -----
```

In [36]:
```
SELECT = np.asarray(SELECT)
assert SELECT.shape == (1000,) and len(set(SELECT.tolist())) == 1000 and SEL
assert SELECT_WHY
m_sel, e = train_with(POOL_X[SELECT], POOL_Y[SELECT])
FIX.append({'ліки': '+1000 відібраних фото з пулу', 'вал': e}); pd.DataFrame
```

Out[36]:

	ліки	вал
0	база: модель B	14.13
1	+1000 випадкових фото з пулу	11.07
2	+1000 відібраних фото з пулу	12.07

Завдання 8.2. Синтез

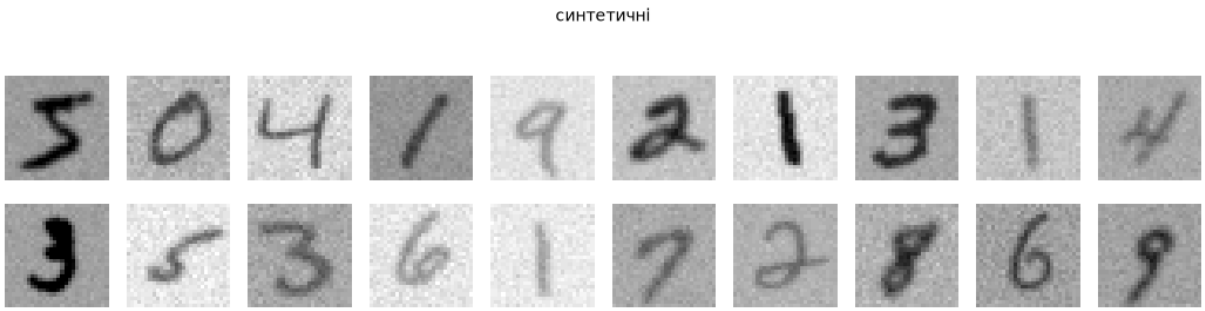
Підберіть параметри `distort` так, щоб синтетичні фото були схожі на **ваші** телефонні — насамперед за категоріями-пріоритетами. Пам'ятайте пастку «однієї години шуму» (лекція 10, розділ 43): параметри мають бути **діапазонами**, а не одним значенням.

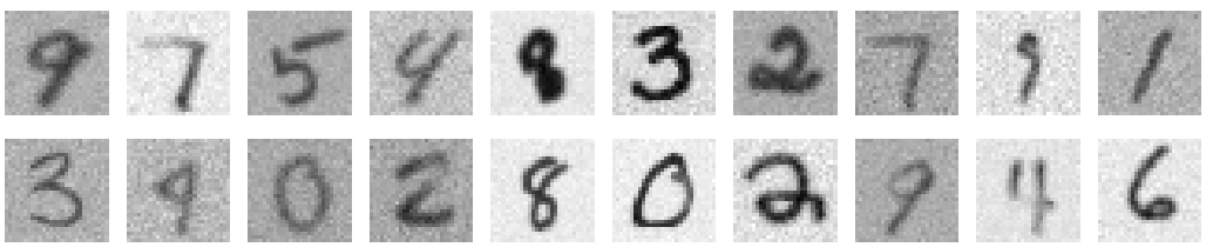
```
SYN = dict(angle=(15, 30), blur=(0.4, 0.8), background=(0.05,
0.15), contrast=(0.7, 0.9), noise=(0.02, 0.05))
SYN_SHARE = 0.6     # частка сканів, які спотворюємо (решта
лишається чистою)
```

Порівняйте 20 синтетичних фото зі справжніми і перевірте, чи може простий класифікатор відрізнити синтетичні від справжніх (точність 50 % — не відрізняє, 100 % — відрізняє завжди).

In [37]:
```
# ----- ВАШІ ВІДПОВІДІ -----
# діапазони, а не одне значення; значення підібрані за image_stats телефонни
# (фон 0,07–0,43; контраст 0,22–0,89; різкість ≈ 5 разів нижча за скани) — н
# розподіл нахилу в сканів і телефонних фото однаковий.
SYN = dict(blur=(0.6, 1.1), background=(0.07, 0.43), contrast=(0.25, 0.9), r
SYN_SHARE = 1.0
# ----- КІНЕЦЬ -----
```

In [38]:
```
assert SYN, 'задайте параметри синтезу'
SYN_X = distort(SCAN_X, seed=42, share=SYN_SHARE, **SYN)
show_grid(SYN_X[:20], title='синтетичні'); show_grid(PHONE_X[:20], title='сп
Xd = np.concatenate([flat(PHONE_X[:1500]), flat(SYN_X[:1500])]); yd = np.r_[
SYN_DISCR = round(100 * float(cross_val_score(LogisticRegression(max_iter=50
print('класифікатор відрізняє синтетичні фото від справжніх із точністю', SY
m_syn, e = train_with(SYN_X, SCAN_Y)
FIX.append({'ліки': '+20 000 синтетичних фото', 'вал': e}); pd.DataFrame(FIX
```





```
класифікатор відрізняє синтетичні фото від справжніх із точністю 60.2 %
```

Out[38]:

	ліки	вал
0	база: модель B	14.13
1	+1000 випадкових фото з пулу	11.07
2	+1000 відібраних фото з пулу	12.07
3	+20 000 синтетичних фото	10.53

In [39]:
```
# калібрування синтезу: середні вимірювання image_stats для сканів, справжні
def _st(X):
    d = pd.DataFrame(image_stats(X[:600])); d['фон'] = [float(np.median(x))
    return d.mean().round(3)
pd.DataFrame({'скани': _st(SCAN_X), 'телефонні': _st(PHONE_X), 'синтетичні':
```

Out[39]:

	скани	телефонні	синтетичні
контраст	0.994	0.537	0.483
різкість	0.072	0.015	0.011
нахил_осі	-11.881	-7.863	-11.619
шум	0.067	0.041	0.036
частка_чорнила	0.127	0.127	0.134
фон	0.000	0.209	0.279

Завдання 8.3. Остаточна модель і план

Зберіть остаточну модель з того, що спрацювало (одна тисяча куплених фото + синтез за бажанням), і **один раз** оцініть її на тестовій вибірці.

In [40]:
```
# ----- ВАШІ ВІДПОВІДІ -----
FINAL_PURCHASE = 'відібрані'    # 'випадкові' / 'відібрані' / '' (без покупк
FINAL_SYNTH = True     # True / False — чи додавати синтез
# ----- КІНЕЦЬ -----
```

In [41]:
```
assert FINAL_PURCHASE in ('випадкові', 'відібрані', '') and FINAL_SYNTH in (
parts_X, parts_y = [], []
if FINAL_PURCHASE:
    ids = rand_idx if FINAL_PURCHASE == 'випадкові' else SELECT
    parts_X.append(POOL_X[ids]); parts_y.append(POOL_Y[ids])
if FINAL_SYNTH:
    parts_X.append(SYN_X); parts_y.append(SCAN_Y)
FINAL, e = train_with(np.concatenate(parts_X), np.concatenate(parts_y)) if p
FIX.append({'ліки': f'остаточна: покупка «{FINAL_PURCHASE or "—"}», синтез {
E_FINAL_TEST = evaluate_on_test(FINAL, 'остаточна модель')
print('остаточна модель: валідаційна', e, '%, тестова', E_FINAL_TEST, '%')
pd.DataFrame(FIX)
```

```
остаточна модель: валідаційна 10.0 %, тестова 10.2 %
```

Out[41]:

	ліки	вал
0	база: модель B	14.13
1	+1000 випадкових фото з пулу	11.07
2	+1000 відібраних фото з пулу	12.07
3	+20 000 синтетичних фото	10.53
4	остаточна: покупка «відібрані», синтез так	10.00

In [42]:
```
# ----- ВАШІ ВІДПОВІДІ -----
PLAN = [
    '1. Замінити виробничу PCA(8)+LR на MLP(256): валідаційна 32,2 → 14,13 %
    '2. Не докуповувати скани, а навчати на суміші: розбіжність моделі B — 9
    '3. Використати бюджет 1000 фото (відбір за контрастом і впевненістю) +
    '4. Перевірити й виправити мітки валідаційної вибірки (22 підозрілі з пе
    '5. Коли бюджет вичерпано: розширити синтез тьмяності (залишок стелі ≈ 7
]
# ----- КІНЕЦЬ -----
```

In [43]:
```
assert len(PLAN) >= 3, 'план має містити щонайменше три кроки'
for k, s in enumerate(PLAN, 1):
    print(k, s)
```

```
1 1. Замінити виробничу PCA(8)+LR на MLP(256): валідаційна 32,2 → 14,13 %
(−18,07 п. п.), бо діагноз був «відхилення» (19,4 п. п.) — таблиця дій, роз
д. 5.
2 2. Не докуповувати скани, а навчати на суміші: розбіжність моделі B — 9,73
п. п. (14,13 − 4,4), дисперсія лише 3,63 — розд. 6.
3 3. Використати бюджет 1000 фото (відбір за контрастом і впевненістю) + син
тез 20 000 фото distort з діапазонами: очікується зниження 14,13 → ≈10,0 %
(−4,1 п. п.), що менше за стелю тьмяності 7,21 п. п.; тестова похибка 10,2 %
— розд. 7–8.
4 4. Перевірити й виправити мітки валідаційної вибірки (22 підозрілі з перши
х 100): виміряна похибка 14,13 → 12,73 % (−1,40 п. п.) без зміни моделі; пот
ім розмітити решту 112 помилок — розд. 7.
5 5. Коли бюджет вичерпано: розширити синтез тьмяності (залишок стелі ≈ 7,21
− 4,1 ≈ 3,1 п. п.), додати аугментацію поворотами проти нахилу (стеля 2,12
п. п.) і покращити локалізатор A (влучає лише у 82,1 % випадків, 195 із 452
помилок системи) — розд. 7, 9.
```

9. Аналіз помилок за частинами

Сервіс отримує не вирізану цифру, а фото бланка 56 × 56: цифра може бути будь-де, поруч — плями (лекція 10, розділи 53–54). Конвеєр:

- **A** — локалізатор без навчання (`locate`): знаходить цифру і вирізає 28 × 28;
- **C′** — класифікатор, навчений на суміші і на копіях, зсунутих на 1–2 пікселі.

Тест книжки: підставте замість виходу A **ідеальний** виріз (справжнє положення цифри). Якщо відповідь стала правильною — помилка на совісті A, якщо ні — C′.

In [44]:
```
CANVAS = INFO['полотна']
CX, CPOS = make_canvas(DEV_X, 500 + VARIANT, blobs=tuple(CANVAS['плями']), l
t0 = time.time()
Cp = model_b().fit(np.concatenate([XB_tr, flat(shifted_copies(MIX_X[mix_tr][
POS_A = locate(CX)
print(f'C′ навчено за {time.time() - t0:.0f} с')
fig, axs = plt.subplots(1, 5, figsize=(12, 2.6))
for k, ax in enumerate(axs):
    ax.imshow(CX[k], cmap='gray_r', vmin=0, vmax=1)
    for (r0, c0), col in [(CPOS[k], C3), (POS_A[k], C2)]:
        ax.add_patch(plt.Rectangle((c0 - 0.5, r0 - 0.5), 28, 28, fill=False,
    ax.set_title(f'цифра {DEV_Y[k]}', fontsize=9); ax.axis('off')
fig.suptitle('зелена рамка — справжнє положення, помаранчева — вихід A', for
```

```
C′ навчено за 7 с
```



Завдання 9.1

Допишіть три рядки: прогноз системи на вирізах A, прогноз C′ на ідеальних вирізах і маску помилок. Функції: `crop(CX, POS_A)`, `crop(CX, CPOS)`, `flat(...)`, `Cp.predict(...)`.

In [45]:
```
# ----- ВАШІ ВІДПОВІДІ -----
pred_sys = Cp.predict(flat(crop(CX, POS_A)))      # прогноз системи A → C′
pred_ideal = Cp.predict(flat(crop(CX, CPOS)))     # прогноз C′ на ідеальних
wrong_sys = pred_sys != DEV_Y                     # True там, де система пом
# ----- КІНЕЦЬ -----
```

In [46]:
```
blame_A = int((wrong_sys & (pred_ideal == DEV_Y)).sum()); blame_C = int((wrc
PARTS = {'похибка_системи': round(100 * float(wrong_sys.mean()), 2), 'помилс
         'A_влучив': round(100 * float((np.abs(POS_A - CPOS) <= 2).all(1).me
PARTS
```

Out[46]:
```
{'похибка_системи': 30.13,
 'помилок': 452,
 'вина_A': 195,
 'вина_C': 257,
 'A_влучив': 82.1}
```

In [47]:
```
# ----- ВАШІ ВІДПОВІДІ -----
PARTS_VERDICT = 'C'     # 'A' або 'C' — який компонент покращувати першим
PARTS_WHY = 'Система помиляється у 30,13 % (452 фото з 1500). З них вина A -
# ----- КІНЕЦЬ -----
```

In [48]:
```
assert PARTS_VERDICT in ('A', 'C') and PARTS_WHY
```

10. Звіт колеги на критику

Колега з іншої команди розв'язував ту саму задачу на своїх даних (варіанта з такими числами в групі немає) і надіслав звіт. У ньому **сім** методичних помилок. Знайдіть щонайменше **п'ять**: для кожної — номер пункту, у чому помилка і як треба було зробити (з посиланням на розділ книжки).

> **Звіт: модель читання цифр із фото, версія 2**
>
> **1. Поділ даних.** Перемішав усі 26 000 зображень (20 000 сканів і 6000 фото) і випадково відклав по 10 % на валідаційну і тестову вибірки.
>
> **2. Людський рівень.** Я і троє колег підписали по 50 фото; середня похибка — 6 %, найкраща — 1 %. Модель на валідаційній вибірці помиляється у 2,4 % випадків, тобто вже краща за людей: усувного відхилення немає, працювати над моделлю далі немає сенсу.
>
> **3. Діагноз.** На фото, які надсилають клієнти, похибка — 11,8 %; на навчально-валідаційній вибірці — 2,9 %, на навчальній — 0,3 %. Це перенавчання (висока дисперсія). Посилив регуляризацію (alpha = 1): навчальна 2,1 %, фото клієнтів — 12,6 %. Дивно, що не допомогло.
>
> **4. Аналіз помилок.** Переглянув 100 помилок на фото клієнтів: 40 із них розмиті. Якщо усунути розмиття, похибка зменшиться на 40 % — до 0,4 %. Тому розмиття — пріоритет номер один.
>
> **5. Синтез.** Сфотографував чистий аркуш у кафе і наклав цей самий фон на всі 20 000 сканів. На валідаційній вибірці з пункту 1 похибка впала з 2,4 % до 2,1 % — синтез працює.
>
> **6. Вибір моделі.** Порівняв 12 варіантів моделі на тестовій вибірці й обрав найкращий — 9,7 %. Це і є очікувана похибка в клієнтів.
>
> **7. Аналіз за частинами.** Підставив замість відповіді класифікатора правильну цифру — і система стала правильною в 100 % випадків. Отже, усі помилки — від класифікатора, локалізатор не винен.

In [49]:
```
# ----- ВАШІ ВІДПОВІДІ -----
CRITIQUE = [
    {'пункт': 1, 'помилка': 'Перемішав скани й фото і з суміші відклав валід
    {'пункт': 2, 'помилка': 'Порівняв модель з усередненим людським рівнем (
    {'пункт': 3, 'помилка': 'Розрив 11,8 % − 2,9 % = 8,9 п. п. між фото клієн
    {'пункт': 4, 'помилка': '40 розмитих із 100 помилок: «усунення дасть −40
    {'пункт': 5, 'помилка': "Синтез: один чистий аркуш з однієї кав'ярні нак
    {'пункт': 6, 'помилка': '12 варіантів моделі порівнював на тестовій вибі
```

```
    {'пункт': 7, 'помилка': 'Підставив замість відповіді класифікатора прави
]
# ----- КІНЕЦЬ -----
```

In [50]:
```
assert len(CRITIQUE) >= 5 and all(c.get('помилка') and c.get('як_правильно')
pd.DataFrame(CRITIQUE)
```

Out[50]:

	пункт	помилка	як_правильно
0	1	Перемішав скани й фото і з суміші відклав валідаційну та тестову вибірки: вони на 77 %...	Валідаційна і тестова вибірки — з цільового розподілу (лише телефонні фото); у навчанн...
1	2	Порівняв модель з усередненим людським рівнем (6 %) і з сумішшю у валідації й вирішив,...	Людський рівень — найкращий результат (1 %), порівнювати на тих самих фото з цільового...
2	3	Розрив 11,8 % – 2,9 % = 8,9 п. п. між фото клієнтів і навч-вал назвав перенавчанням (д...	Великий розрив вал – навч-вал — це розбіжність даних; ліки — дані, схожі на фото (доку...
3	4	40 розмитих із 100 помилок: «усунення дасть –40 % похибки, до 0,4 %». Це арифметична п...	Стеля = частка помилок × загальна похибка (0,4 × 11,8 ≈ 4,7 п. п.), і це верхня межа; ...
4	5	Синтез: один чистий аркуш з однієї кав'ярні накладений на всі 20 000 сканів — модель в...	Параметри синтезу — діапазони (різні фони, контрасти, розмиття), перевіряти на валідац...
5	6	12 варіантів моделі порівнював на тестовій вибірці й обрав найкращий (9,7 %): тест ста...	Обирати модель на валідаційній вибірці, до тестової звертатися один раз для остаточної...
6	7	Підставив замість відповіді класифікатора правильну цифру і отримав 100 % — це вірно з...	Підставляти ідеальний вихід кожного компонента окремо: ідеальний виріз замість виходу ...

Частина 3. Звіт (0,5 бала)

---

11. Звіт

Відповідайте **числами з цього ноутбука**, а не загальними словами.

1. **Діагноз виробничої моделі.** Назвіть три похибки і три складові розкладу. Чому навчально-валідаційну вибірку взято саме з того джерела?
2. **Людський рівень.** Ваша похибка на 50 фото і найкраща в групі для вашого варіанта. Чи змінив би діагноз людський рівень 0 %? А 5 %?
3. **Крива навчання.** Чи допоможуть дані того самого джерела? Що на графіку це показує?

4. **Дії.** Які дві дії допомогли, на скільки п. п.? Що сталося з контрольною дією і чому?
5. **Модель B.** Яка складова її похибки головна і чому більше сканів її не зменшить?
6. **Аналіз помилок.** Три пріоритети зі стелями. Яка частка похибки — хибні мітки? Чи варто виправляти мітки валідаційної вибірки?
7. **Ліки.** Що дало найбільший виграш і чи вклався він у стелю з пункту 6? Наскільки синтетичні фото відрізняються від справжніх?
8. **Тест.** Валідаційна і тестова похибки остаточної моделі. Скільки разів ви зверталися до тестової вибірки?
9. **Аналіз за частинами.** Хто винен частіше — A чи C′ — і що ви зробите першим?
10. **Звіт колеги.** Яка з його помилок найдорожча для замовника і чому?

In [51]:
```
# ----- ВАШІ ВІДПОВІДІ -----
REPORT = '\n1. Діагноз виробничої моделі PCA(8)+LR (навчена на суміші): навч
# ----- КІНЕЦЬ -----
```

Декларація про використання штучного інтелекту

Наприкінці звіту зазначте, чи користувались ШІ-асистентом, для чого саме і що перевіряли самостійно. Використання дозволене; не дозволено видавати чуже за своє.

In [52]:
```
def _num(d):
    return {k: (round(float(v), 4) if isinstance(v, (int, float, np.floating


result = {
    'студент': STUDENT_NAME, 'номер': LIST_NUMBER, 'варіант': VARIANT,
    'версія_sklearn': sklearn.__version__,
    'джерело_навч_вал': TRAIN_SOURCE, 'чому_навч_вал': WHY_TRAIN_DEV,
    'людина': {'відповіді': HUMAN_ANSWERS, 'похибка': HUMAN_ERR},
    'виробнича': {'похибки': _num(E_PROD), 'розклад': _num(DECOMP_PROD), 'ді
    'крива': CURVE.to_dict('records'), 'більше_тих_самих_даних': MORE_SAME_D
    'дії': ACT_TABLE.round(2).to_dict('records'), 'дії_висновок': ACTIONS_CO
    'B': {'похибки': _num(E_B), 'розклад': _num(DECOMP_B), 'діагноз': DIAGNO
          'за_виправленими_мітками': E_B_FIXED, 'виправлено_міток': len(rela
    'аналіз_помилок': {'мітки': TAGS, 'номери_фото': ERR_IDX.tolist(), 'табл
                       'пріоритет': PRIORITY, 'чому': PRIORITY_WHY},
    'ліки': FIX, 'відбір': {'номери': SELECT.tolist(), 'чому': SELECT_WHY},
    'синтез': {'параметри': {k: list(v) for k, v in SYN.items()}, 'частка':
    'остаточна': {'покупка': FINAL_PURCHASE, 'синтез': FINAL_SYNTH, 'тест':
    'план': PLAN,
    'частини': PARTS, 'частини_висновок': PARTS_VERDICT, 'частини_чому': PAR
    'критика': CRITIQUE, 'звіт': REPORT,
}
fname = 'lr7_результат_%02d.json' % LIST_NUMBER
with open(fname, 'w', encoding='utf-8') as f:
```

```
    json.dump(result, f, ensure_ascii=False, indent=1, default=lambda o: o.i
print('збережено', fname)
```

```
збережено lr7_результат_04.json
```

Перед здачею

- **Kernel → Restart & Run All** — ноутбук має пройти згори вниз без помилок (повний прогін — 10–15 хвилин).
- Усі `assert` зелені.
- Файл `lr7_результат_NN.json` створено; здайте його разом із ноутбуком.
- Тестову вибірку використано **один** раз.
- У звіті — числа, а не загальні формулювання; декларацію про ШІ заповнено.